SQL Injection Penetration Testing Practical Report

Objective of the Practical

The objective of this practical is to understand and demonstrate SQL Injection (SQLi) as a common web application vulnerability. The practical focuses on:

- Understanding how login pages interact with backend databases
- Demonstrating manual SQL injection using true statements
- Demonstrating automated SQL injection using penetration testing tools
- Showing how attackers can bypass authentication and extract database information
- Understanding why secure coding and database hardening are necessary

This practical was conducted strictly in a controlled laboratory environment for educational purposes.

Lab Environment and Setup

The practical was carried out using a virtualized lab environment. Hardware and Virtualization

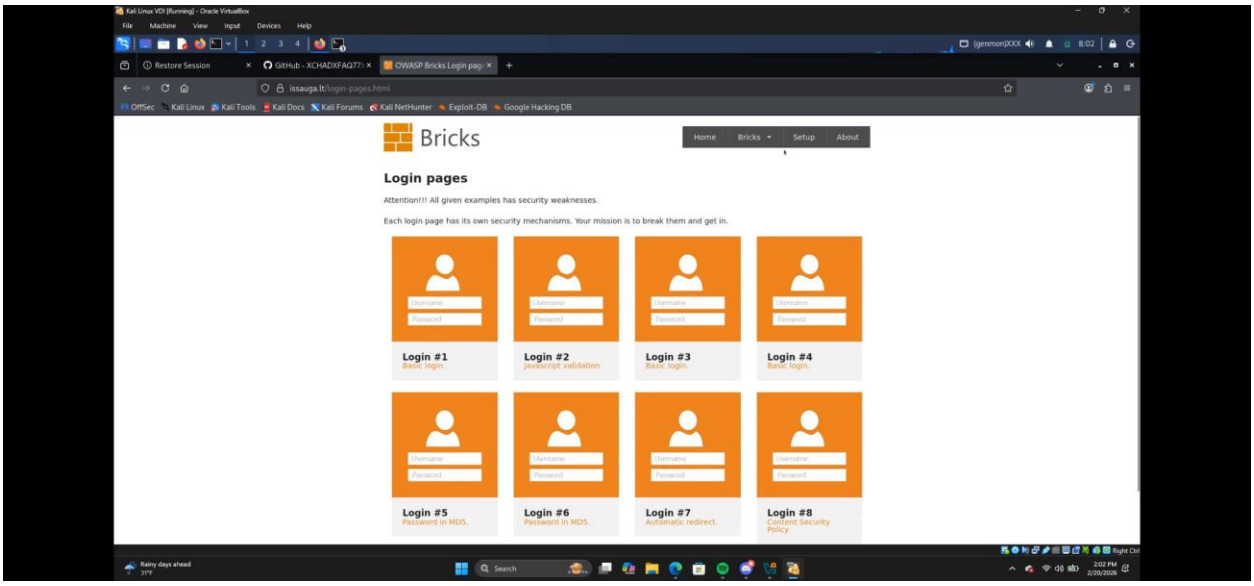
- Host machine running a virtual machine environment
- Internal isolated network

Virtual Machines Used

- Kali Linux (Attacker machine)
- Vulnerable web application server
- Windows machines

Software and Tools

- Kali Linux
- Vulnerable web application
- BurpSuite
- SQLmap
- Web browser (Firefox)



1. Background Theory (Brief)

Web login pages usually validate user credentials by sending input values (username and password) to a backend database using SQL queries.

A typical query looks like:

SELECT * FROM users
WHERE username='admin' AND password='password';

If an attacker can manipulate the input fields, they can change the logic of the SQL query. By injecting a true statement, the database may return a valid response even when credentials are incorrect. SQL Injection can be performed manually or using automated tools.

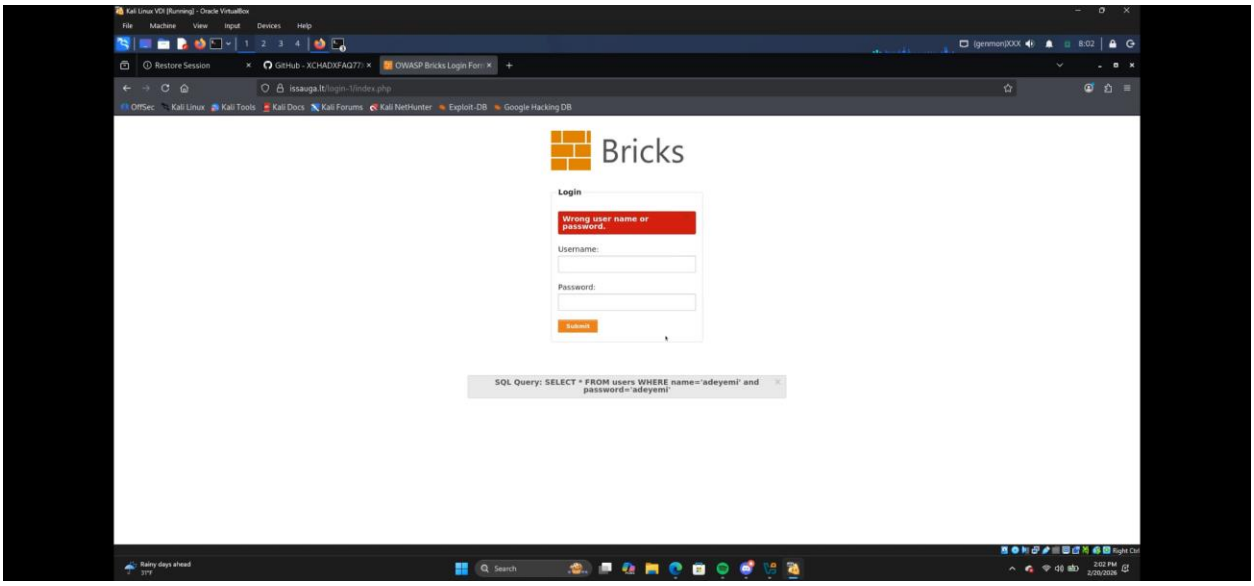
2. Manual SQL Injection Testing

Step 1. Normal Login Attempt

The login page was first tested using invalid credentials.

- Username: admin
- Password: wrongpassword

The application returned an error message indicating incorrect credentials.



This confirms that the login page is connected to a backend database that validates input.

Step 2. Manual SQL Injection Using True Statement

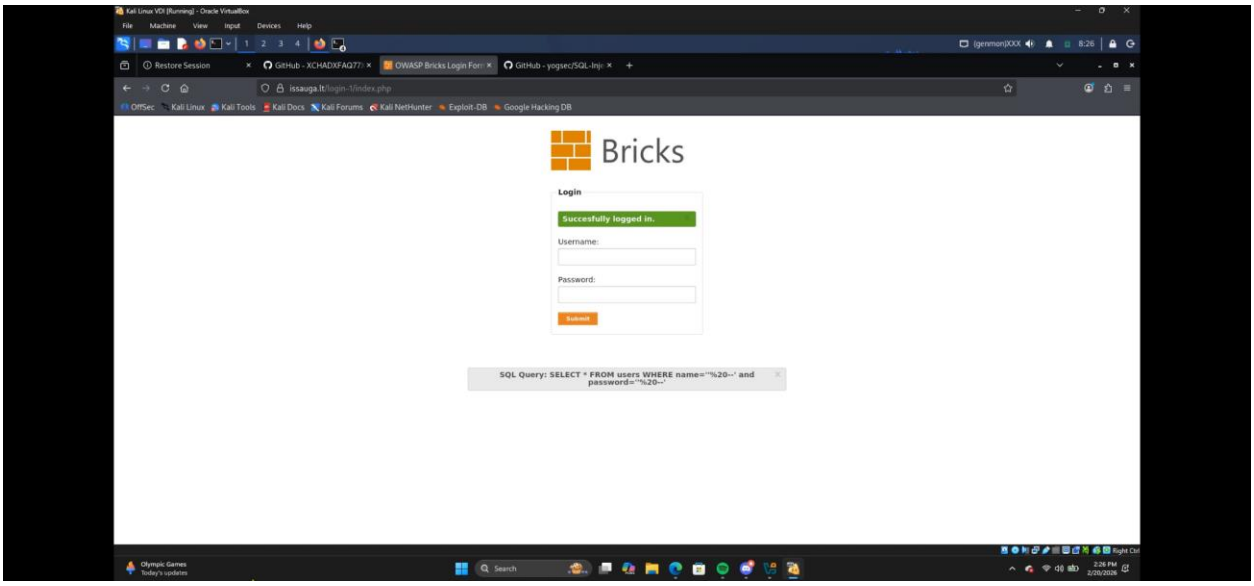
To test for SQL injection vulnerability, a true SQL condition was injected into the login field.

Injected payload:

Username: "%20— '
Password : "%20— '

This payload forces the SQL query to evaluate as true, regardless of stored credentials.

<u>Result</u>
The application returned a successful login response, confirming that authentication was bypassed.



Observation

- The login page does not properly sanitize user input
- SQL logic is directly influenced by user-provided data

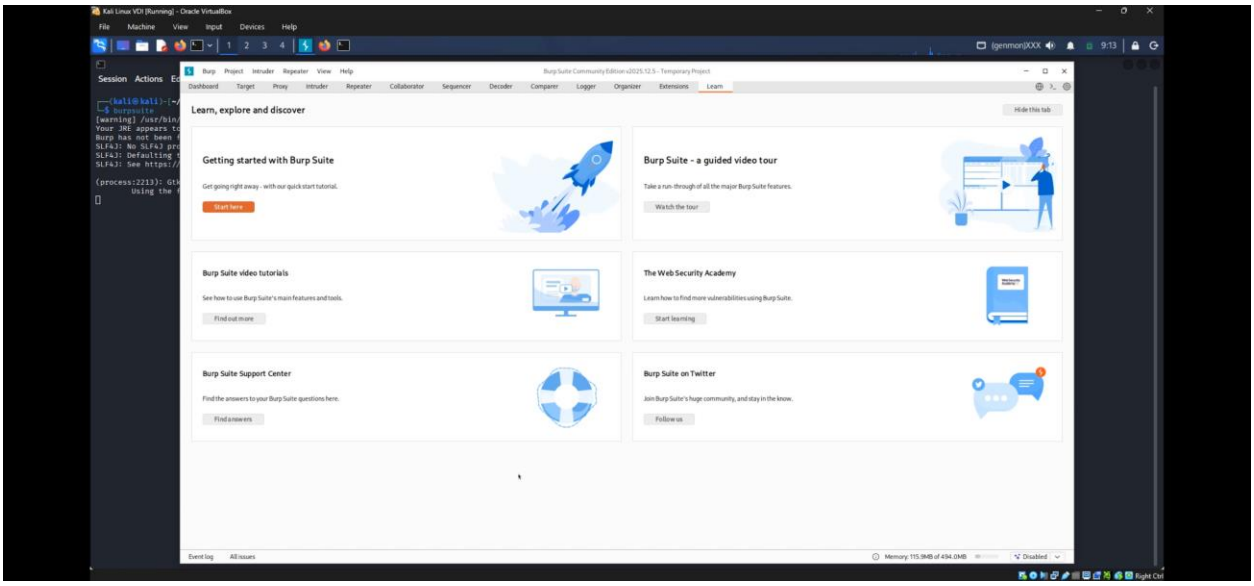
3. Automated SQL Injection Testing

Manual SQL injection is slow and unreliable. Automated tools allow faster and deeper testing.

3A. Capturing Login Request with Burp Suite

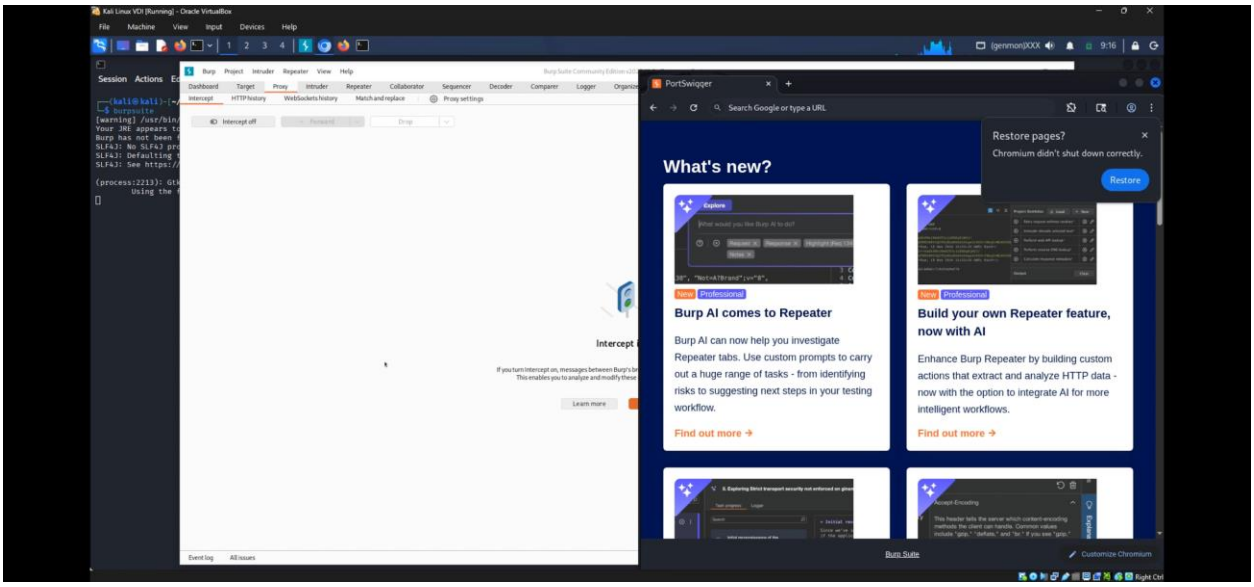
Step 1. Launching Burp Suite

BurpSuite Community Edition was launched from Kali Linux.



Step 2. Proxy Configuration

- Proxy intercept was initially turned off
- Burp's embedded browser was opened
- Login page was accessed through Burp browser

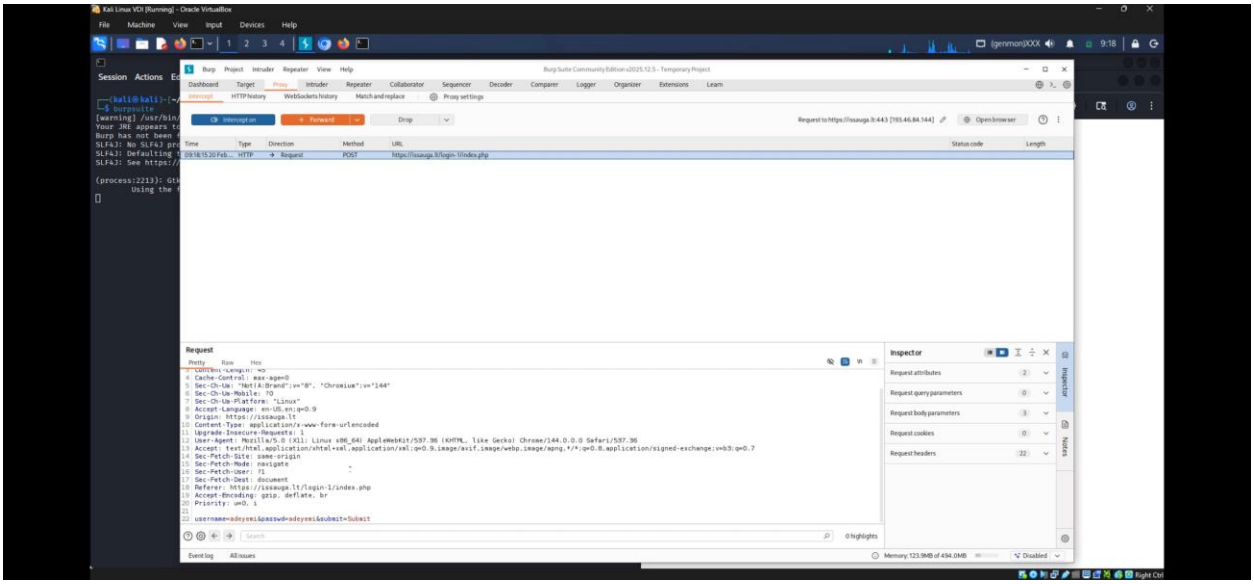


Step 3. Intercepting POST Request

- Dummy credentials were entered into the login form
- Proxy intercept was turned on
- Login request was submitted

Burp intercepted the HTTP POST request containing parameters such as:

- username
- password
- submit

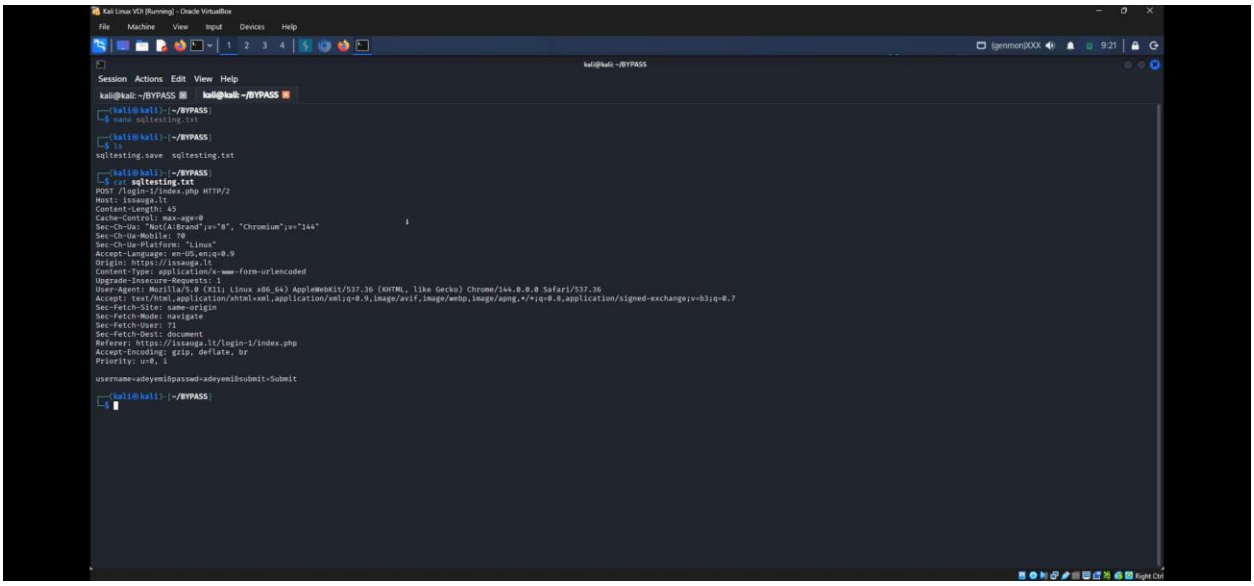


This request represents the exact data sent from the client to the server.

Step 4. Saving Captured Request

The intercepted request was copied and saved into a text file on Kali Linux.

File name: **Sqltesting.txt**



3B. Automated Exploitation Using SQLmap

SQLmap is an automated SQL injection tool used to identify and exploit SQL vulnerabilities.

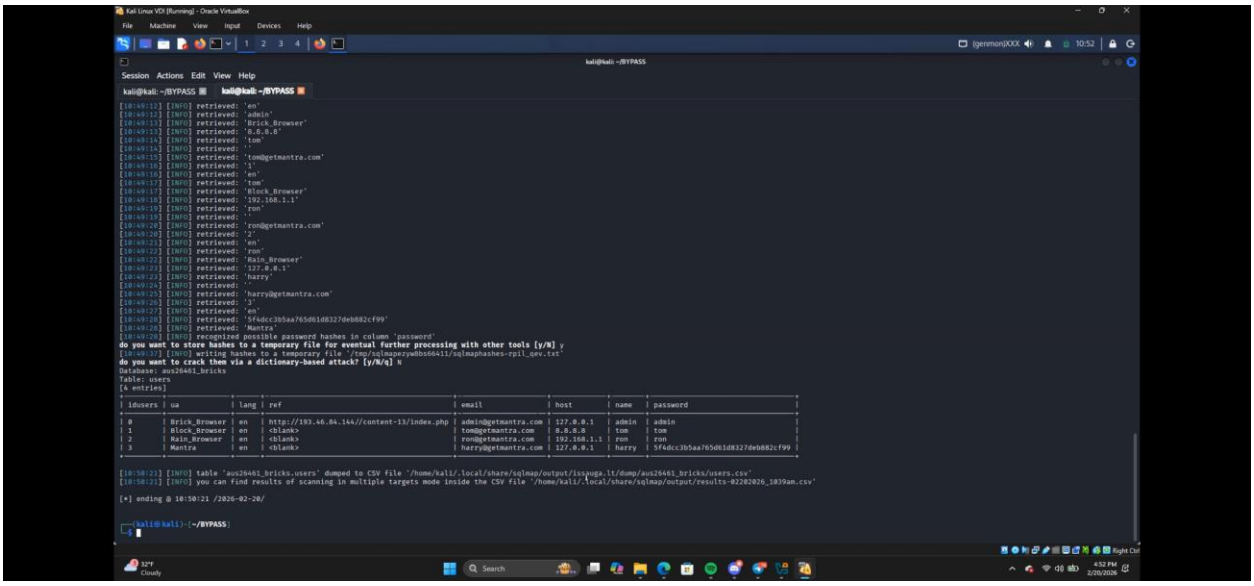
Step 1: Initial SQL Injection Test

SQLmap was executed using the captured request file.

<u>Command used:</u>
sqlmap -l sqltesting.txt -p passwd --dumps

<u>Result</u>

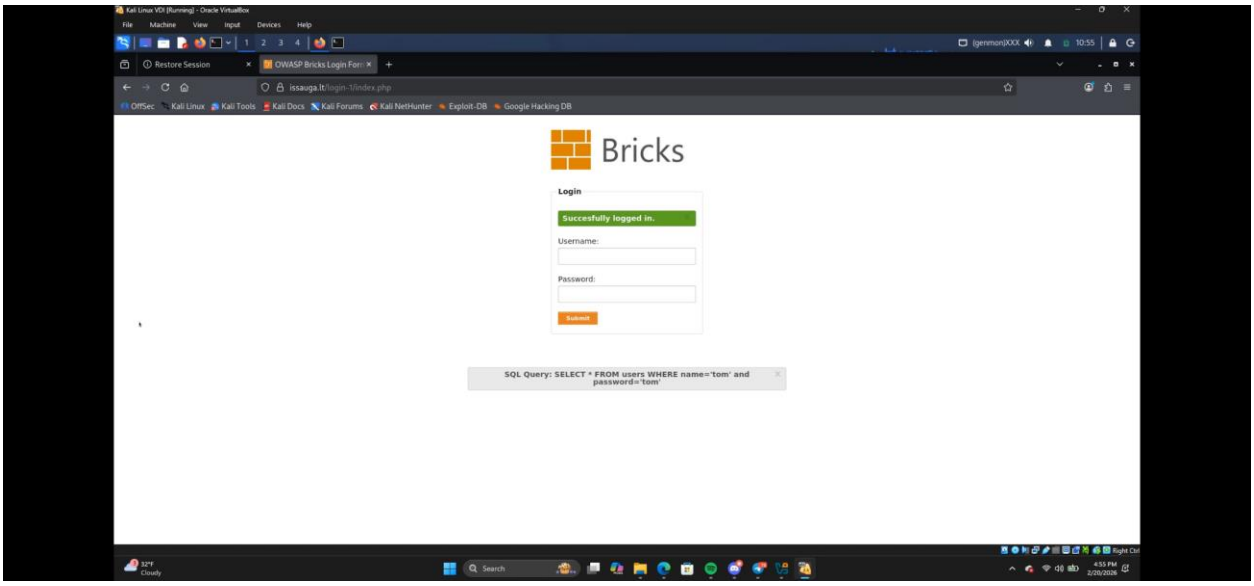
- SQLmap detected SQL injection vulnerability
- Backend database type identified (MySQL)
- Available databases were listed



4. Verification of Extracted Credentials

To confirm the validity of the extracted data:

- One of the dumped credentials was used on the login page
- Login was successful



This confirms complete authentication bypass and data compromise.

5. Security Impact and Risk Analysis

The identified vulnerability allows an attacker to:

- Bypass authentication controls
- Gain unauthorized access to the application
- Extract sensitive user data
- Compromise database confidentiality and integrity

If exploited on a live system, this could result in:

- Data breaches
- Credential theft
- Regulatory violations
- Loss of user trust

6. Defensive Measures and Recommendations

To prevent SQL Injection vulnerabilities, developers should:

- Use prepared statements and parameterized queries
- Implement input validation and sanitization
- Avoid direct SQL query concatenation
- Apply least privilege access to databases
- Use updated and patched database versions
- Deploy web application firewalls (WAFs)
- Conduct regular penetration testing and code reviews

7. Conclusion

This practical demonstrates how SQL Injection vulnerabilities can be exploited both manually and automatically. Using Burp Suite and SQLmap, it was possible to bypass authentication and extract sensitive database information.
The exercise highlights the importance of secure coding practices and proactive security testing during application development.